



ClosingScript

This project may appeal to programmers who dislike the significant indentation in CoffeeScript but otherwise like the language for its simplicity and expressiveness. ***ClosingScript*** addresses this problem by adding closing keywords to CoffeeScript syntax to close multi-line constructs, making whitespace no longer significant. Single-line constructs sometimes called one-liners are unchanged.

ClosingScript is a text preprocessor, not a fork, that regenerates significant indentation using these closing keywords.

A closing keyword consists of the slash followed by the name of its matching multi-line construct.

List of closing keywords:

/if, ***/unless***, ***/while***, ***/until***, ***/for***, ***/switch***, ***/loop***, ***/try***, ***/class***, ***/end*** (function)

Advantages:

- Whitespace is not significant.
- Robust code structure.
- Clear visual termination of multi-line constructs.
- Safe copy-pasting.
- Safe auto-formatting.

Disadvantages:

- Slightly more verbose syntax.

Usage:

The preprocessor outputs to *stdout*. To get JavaScript, the CoffeeScript compiler must be installed. Compiler errors match ***ClosingScript*** sources at *linenumber* level.

Assuming both ***ClosingScript*** and the CoffeeScript compiler are in your path.

To convert and compile to JavaScript in one step:
closing script.coffee | coffee -s -c > script.js

To get a converted copy of a source:
closing script.coffee > script.converted.coffee

To get a formatted copy of a source (only reindented consistently):
closing -f script.coffee > script.formatted.coffee

Syntax

The syntax of a multi-line construct is identical to standard CoffeeScript except for the addition of a closing keyword. The list below is not exhaustive and is provided to give an general idea.

```
functionName = (parameters) -> | =>  
  code  
/end [optional following code]           # examples of following code: /end, timeout  
                                           /end )
```

```
do ->  
  code  
/end
```

```
[variable =] if condition  
  code  
[else if condition  
  code]  
[else  
  code]  
/if
```

```
[variable =] for iterable [guard clauses]  
  code  
/for
```

```
[variable =] while condition [guard clauses]  
  code  
/while
```

```
[variable =] loop  
  code  
/loop
```

```
[variable =] switch [variable]  
  when  
    code  
  [when condition then code]  
  [else  
    code]  
/switch
```

```
class name  
  code  
/class
```

```
try  
  code  
[catch ...  
  code ]  
[finally ...  
  code ]  
/try
```

Geany code editor

In Linux, this command can be added to the Build menu of the Geany code editor. It converts and compiles in one step, with errors falling back at guilty lines in the editor window, as well as error code propagation from the preprocessor and the compiler in the built-in compiler message window.

```
set -o pipefail; closing "%f" | coffee -s -c > "%e".js 2> >(sed 's^[stdin\]/%f/' >&2)
```